Relatório de Testes

API RESTful — Locadora de Veículos

Trabalho Prático — Etapa 3

Testes e Documentação (item 3.3)

Marco Túlio

Tecnologias para Análise e Desenvolvimento de Sistemas

38 testes manuais executados via Swagger UI

github.com/MarcoTulioGr/LocadoraVeiculos

1. Metodologia

Todos os endpoints da API foram testados manualmente através da interface do Swagger, com a aplicação em execução no ambiente de desenvolvimento e o banco SQL Server Express populado pelo script `Scripts/01_povoamento.sql`. Para cada teste, este relatório registra o objetivo, o resultado esperado e a captura de tela da chamada com o retorno obtido.

A sequência de testes foi montada de forma encadeada: em cada entidade, um registro é criado, consultado, alterado e por fim excluído, o que verifica as quatro operações sobre o mesmo dado e devolve o banco ao estado inicial ao término.

Cada captura mostra o bloco Curl (com o método, a URL e o corpo enviado), a Request URL, o código de status devolvido e o corpo da resposta — ou seja, a chamada e o retorno na mesma evidência.

Ambiente	Configuração
Aplicação	ASP.NET Core 8 — https://localhost:7297
Documentação	Swagger UI (Swashbuckle) em /swagger
Banco de dados	SQL Server Express — instância SQLEXPRESS, base LocadoraVeiculosDb
Mapeamento	Entity Framework Core 8, padrão Code First
Massa de dados	5 fabricantes, 5 categorias, 12 veículos, 8 clientes, 12 aluguéis

2. Cobertura

Os 38 testes cobrem a totalidade dos endpoints da API, somados a seis casos de erro que exercitam a validação de dados e o tratamento de exceções.

Conjunto	Testes	Escopo
Fabricantes	01 a 05	CRUD completo
Categorias de Veículo	06 a 10	CRUD completo
Veículos	11 a 15	CRUD completo
Clientes	16 a 20	CRUD completo
Aluguéis	21 a 26	CRUD completo e registro de devolução
Filtros	27 a 32	Consultas com INNER JOIN e LEFT JOIN
Casos de erro	33 a 38	Validação, integridade e regras de negócio

Todos os 38 testes foram executados com sucesso, retornando exatamente o código de status e o conteúdo esperados.

3. Registro dos testes

Fabricantes

CRUD completo da entidade Fabricante.

01	GET	/api/Fabricantes
Objetivo	Listar todos os fabricantes cadastrados.	
Resultado esperado	200 OK com os 5 fabricantes do banco.	

Curl

```
curl -X 'GET' \
  'https://localhost:7297/api/Fabricantes' \
  -H 'accept: application/json'
```

Request URL

```
https://localhost:7297/api/Fabricantes
```

Server response

Code | Details

200

Response body


```
  {
    "id": 4,
    "nome": "Chevrolet",
    "paisOrigem": "Estados Unidos",
    "veiculos": []
  },
  {
    "id": 1,
    "nome": "Fiat",
    "paisOrigem": "Italia",
    "veiculos": []
  },
  {
    "id": 5,
    "nome": "Hyundai",
    "paisOrigem": "Coreia do Sul",
    "veiculos": []
  },
  {
    "id": 3,
    "nome": "Toyota",
    "paisOrigem": "Japao",
    "veiculos": []
  },
  {
    "id": 2,
    "nome": "Volkswagen",
    "paisOrigem": "Alemanha",
    "veiculos": []
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed,30 Sep 2026 13:55:32 GMT
server: Kestrel
```

Responses

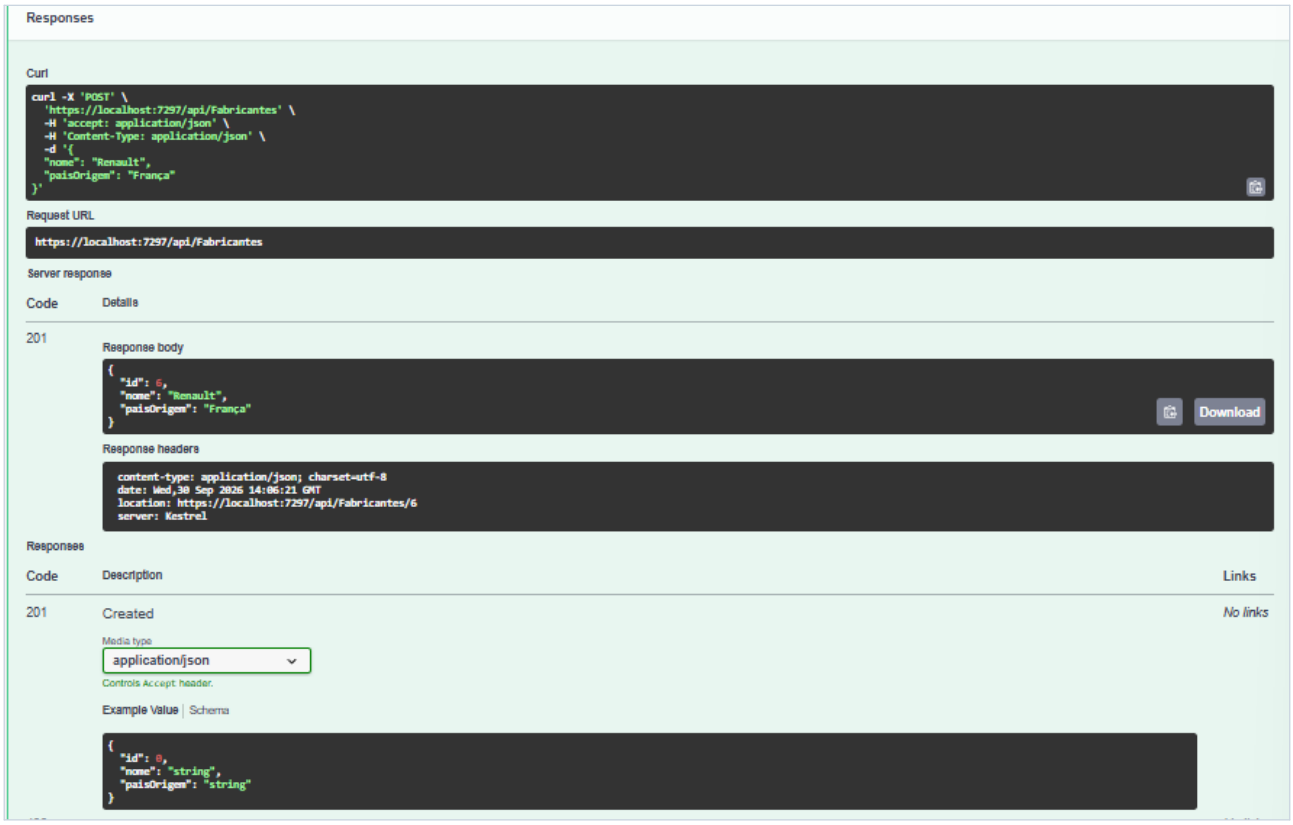
Code	Description	Links
200	OK	*No links*

Media type: application/json

Controls Accept header.

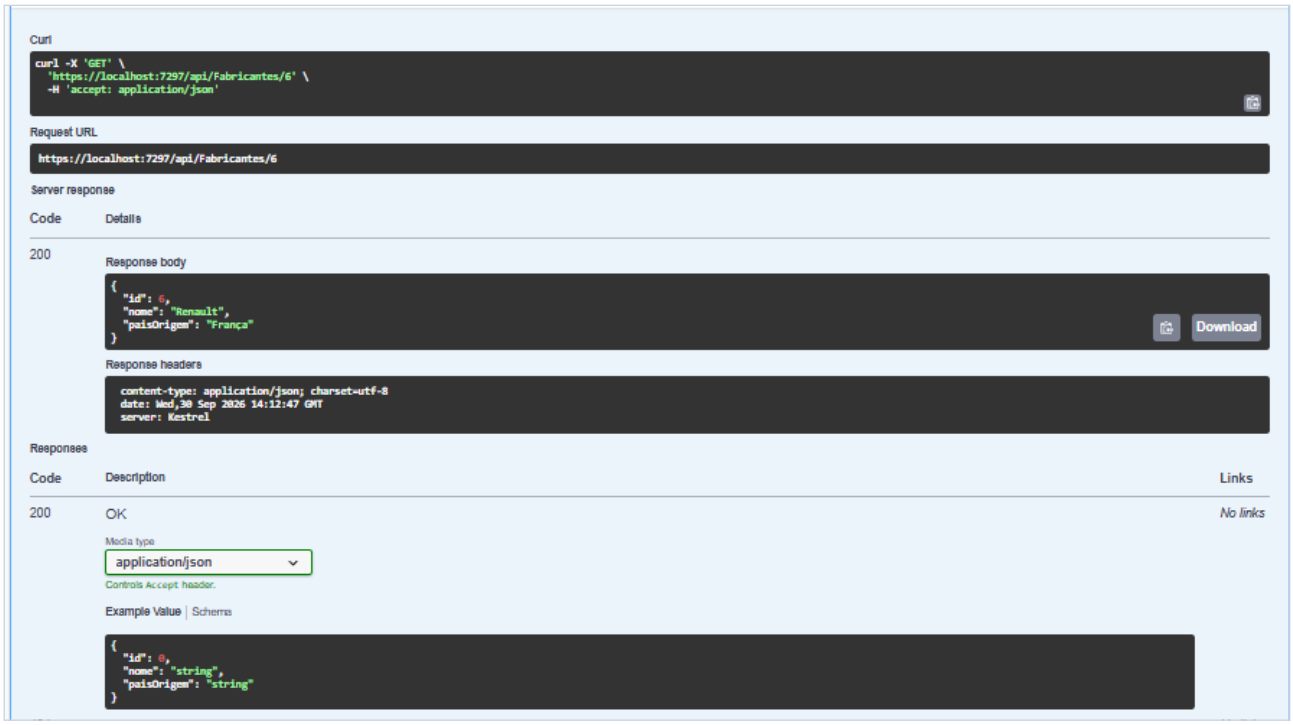
Evidência: 01-fabricantes-get-200.png

02	POST	/api/Fabricantes
Objetivo	Cadastrar o fabricante Renault.	
Resultado esperado	201 Created, com o cabeçalho Location apontando para o novo recurso.	



Evidência: 02-fabricantes-post-201.png

03	GET	/api/Fabricantes/{id}
Objetivo	Consultar o fabricante recém-criado pelo seu id.	
Resultado esperado	200 OK com o registro.	



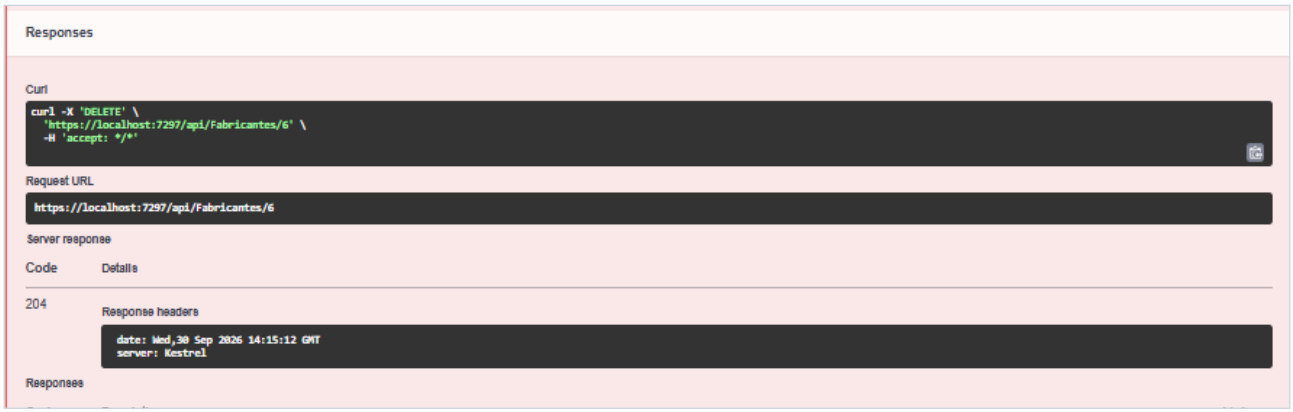
Evidência: 03-fabricantes-getid-200.png

04	PUT	/api/Fabricantes/{id}
Objetivo	Alterar o nome do fabricante.	
Resultado esperado	204 No Content, sem corpo de resposta.	



Evidência: 04-fabricantes-put-204.png

05	DELETE	/api/Fabricantes/{id}
Objetivo	Excluir o fabricante criado no teste 02.	
Resultado esperado	204 No Content — não há veículos vinculados.	

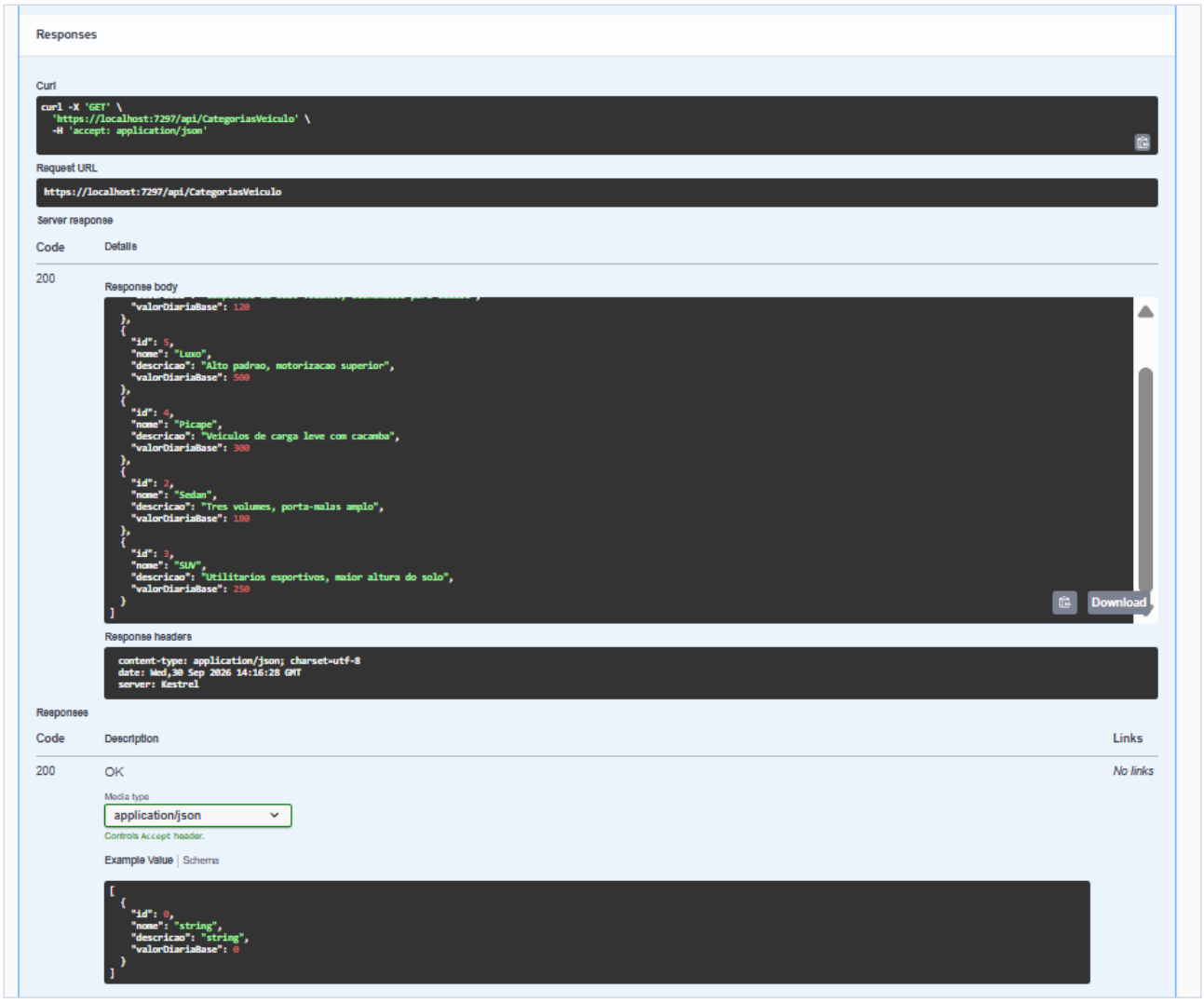


Evidência: 05-fabricantes-delete-204.png

Categorias de Veículo

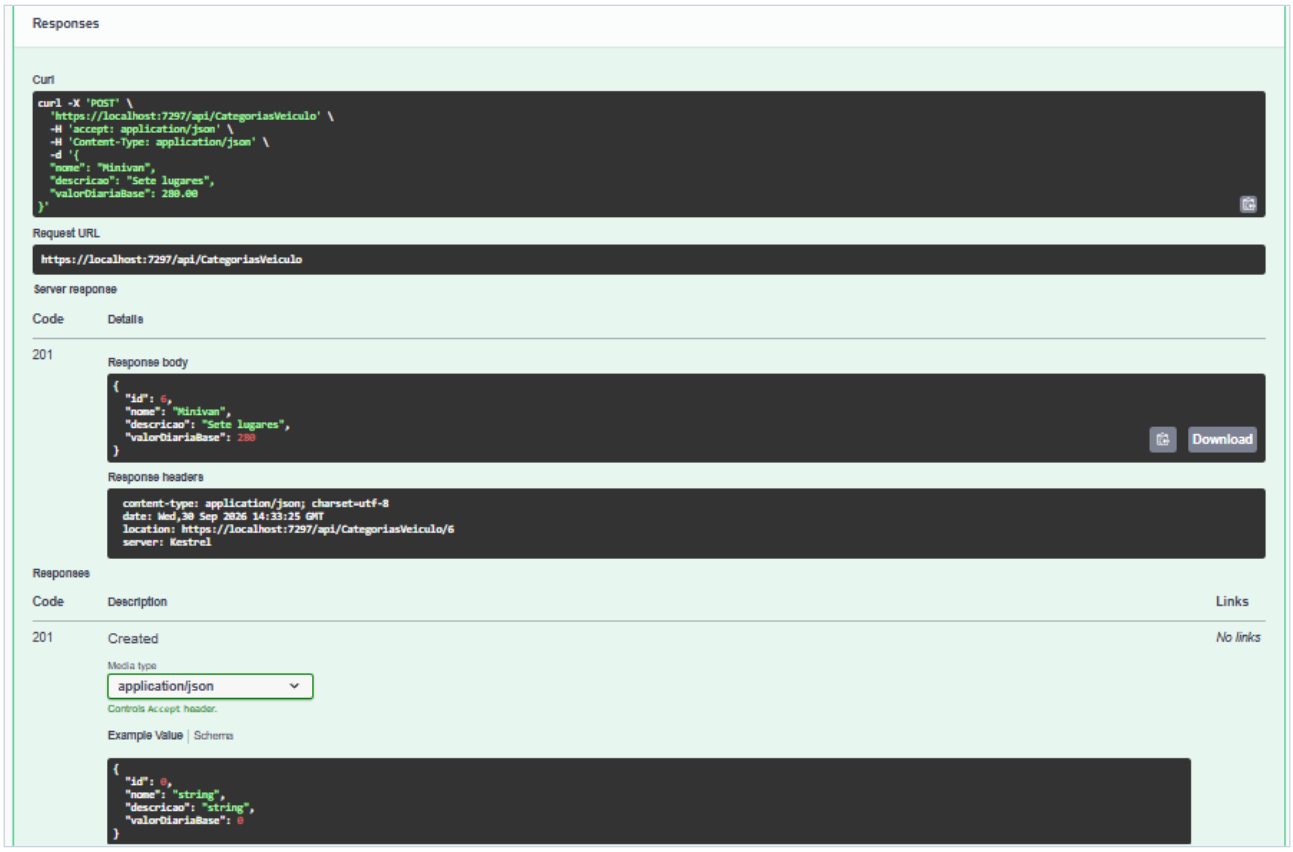
CRUD completo da entidade CategoriaVeiculo.

06	GET	/api/CategoriasVeiculo
Objetivo	Listar as categorias com seus valores de diária.	
Resultado esperado	200 OK com as 5 categorias.	



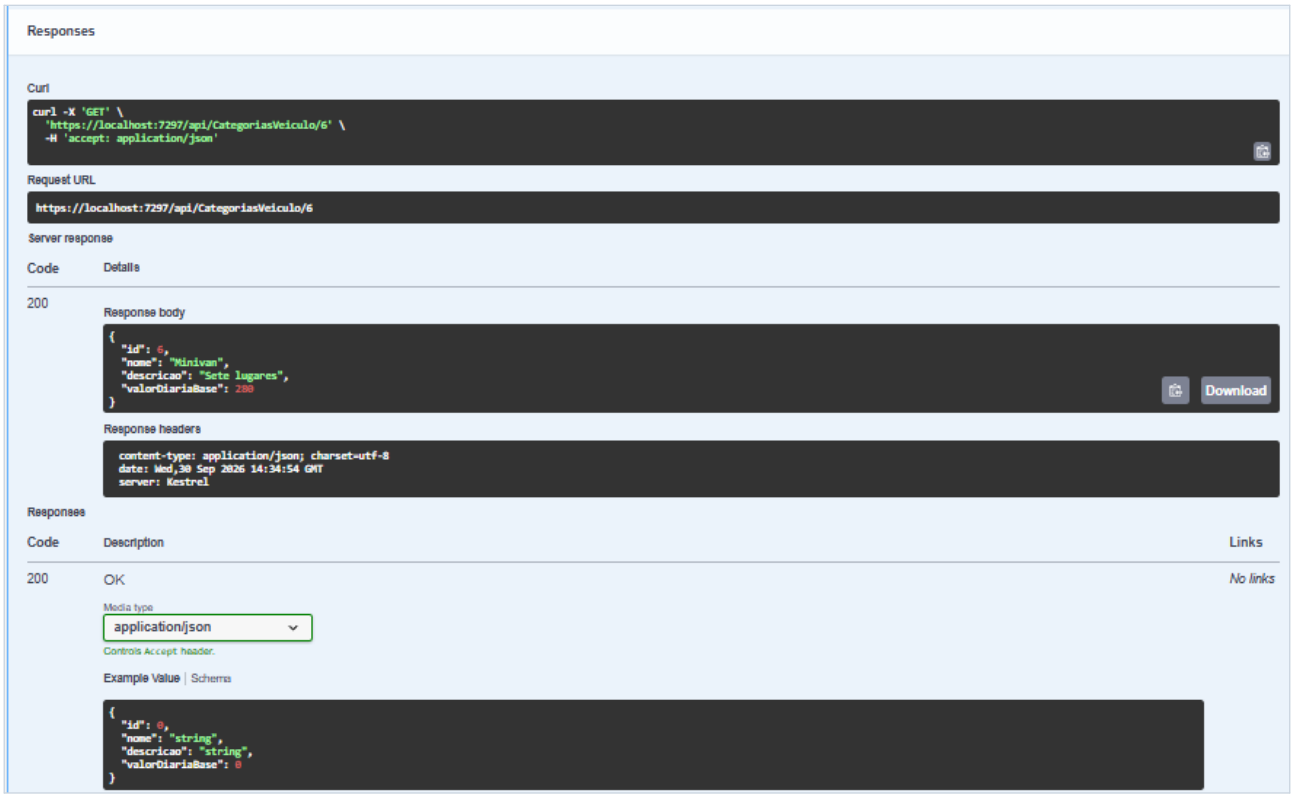
Evidência: 06-categorias-get-200.png

07	POST	/api/CategoriasVeiculo
Objetivo	Cadastrar a categoria Minivan.	
Resultado esperado	201 Created com o id gerado.	



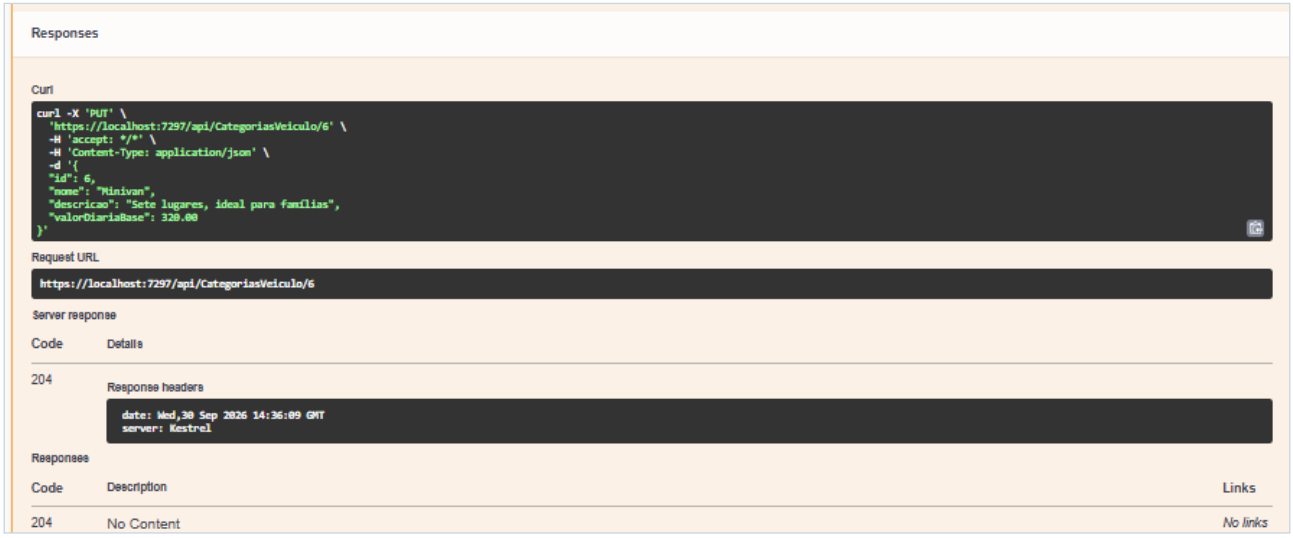
Evidência: 07-categorias-post-201.png

08	GET	/api/CategoriasVeiculo/{id}
Objetivo	Consultar a categoria recém-criada.	
Resultado esperado	200 OK com o registro.	



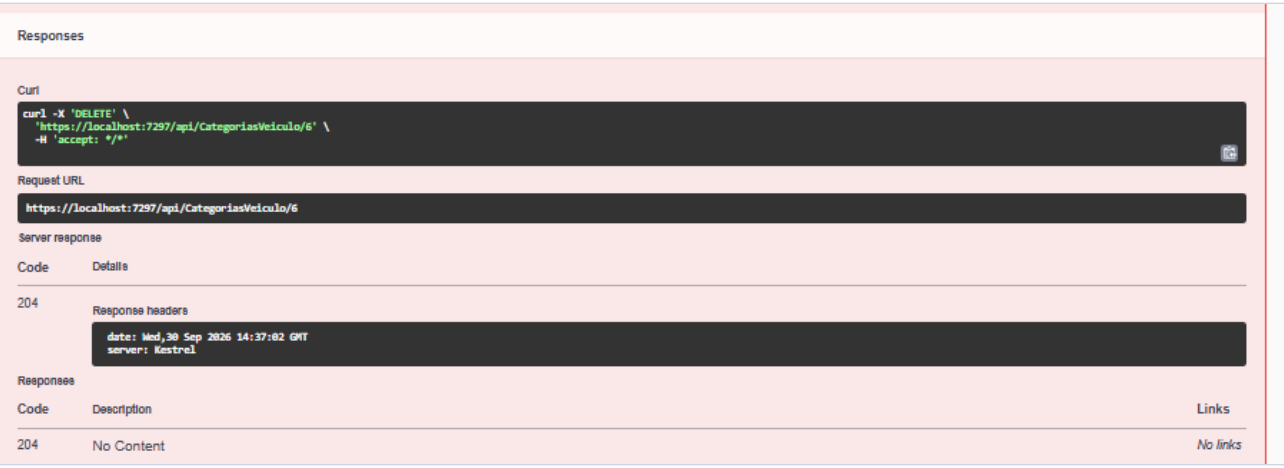
Evidência: 08-categorias-getid-200.png

09	PUT	/api/CategoriasVeiculo/{id}
Objetivo	Reajustar o valor da diária base de 280 para 320.	
Resultado esperado	204 No Content.	



Evidência: 09-categorias-put-204.png

10	DELETE	/api/CategoriasVeiculo/{id}
Objetivo	Excluir a categoria criada no teste 07.	
Resultado esperado	204 No Content — nenhum veículo classificado nela.	

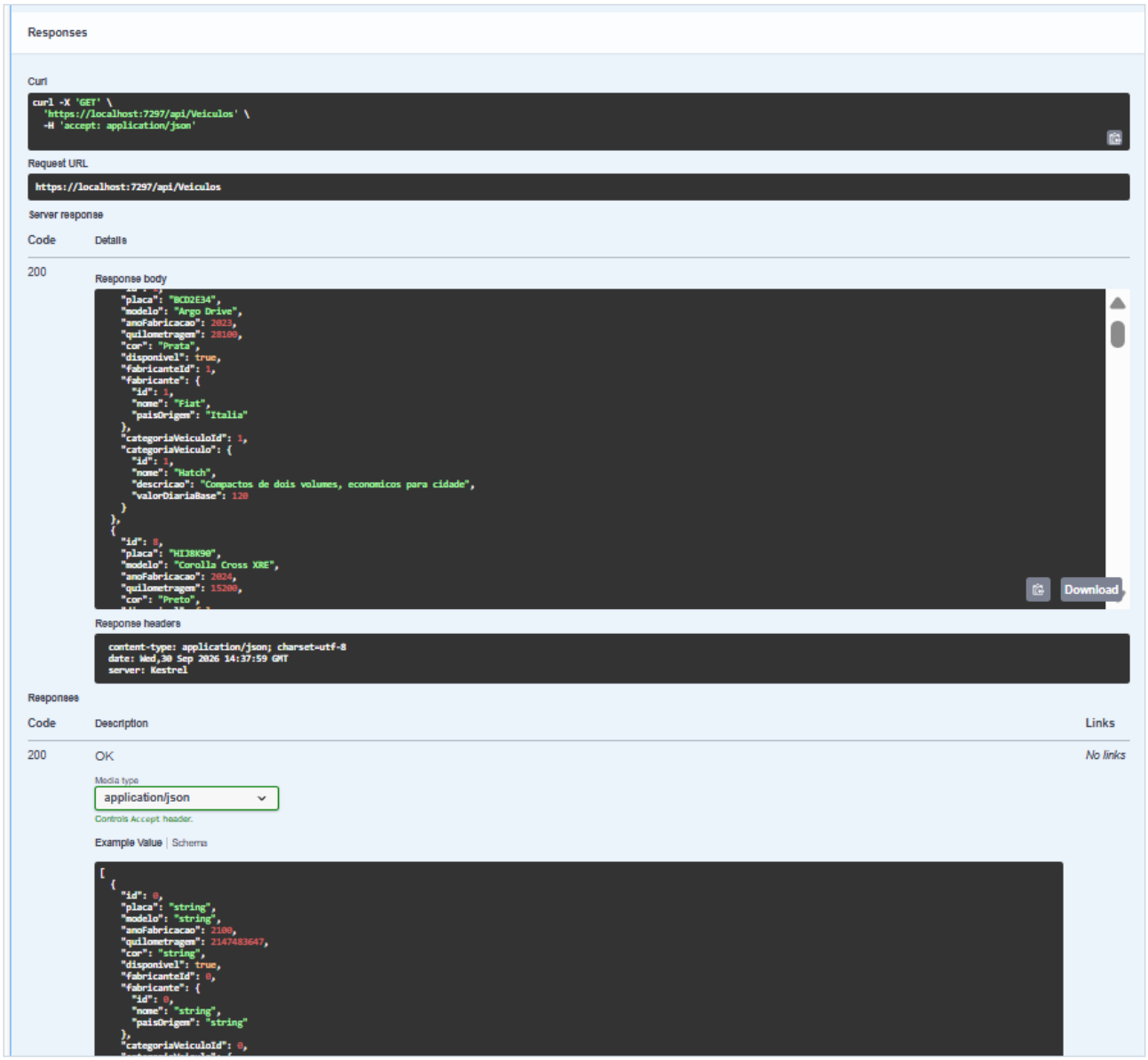


Evidência: 10-categorias-delete-204.png

Veículos

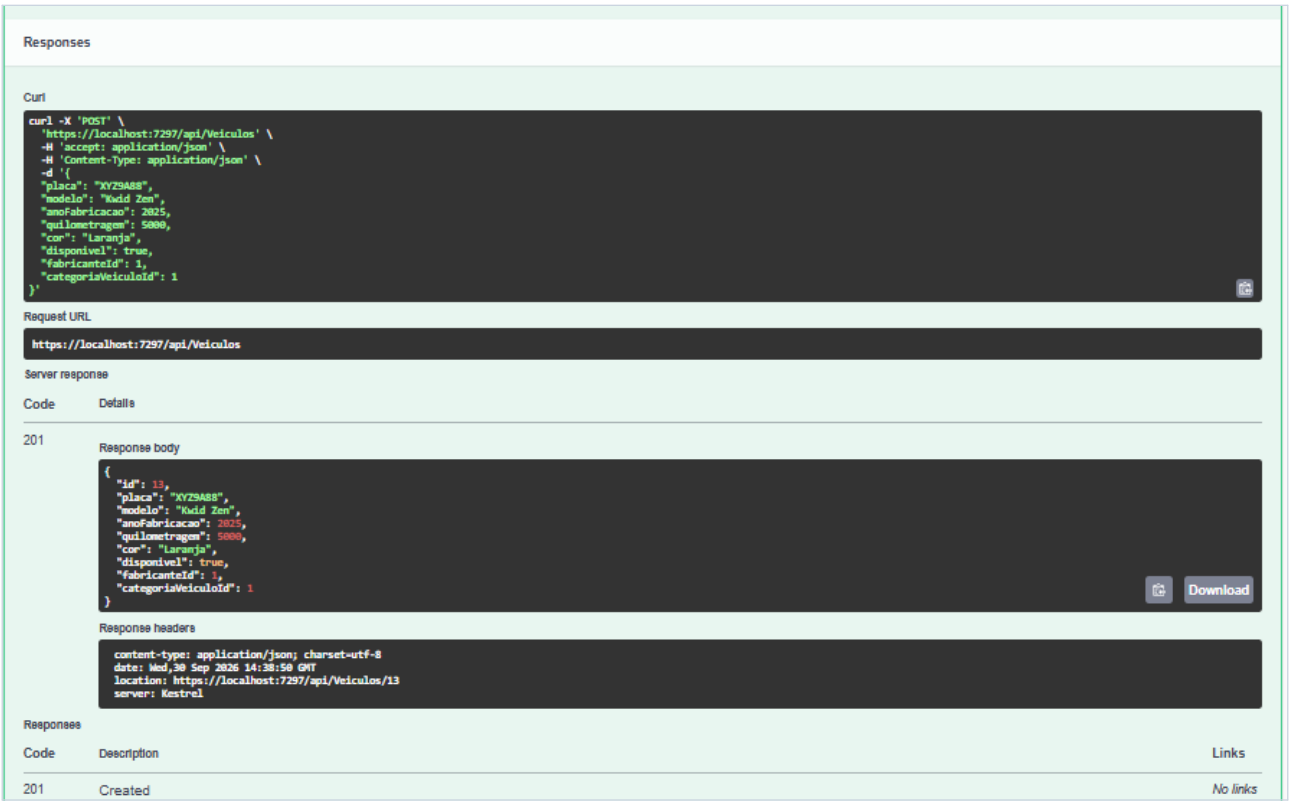
CRUD completo da entidade Veiculo.

11	GET	/api/Veiculos
Objetivo	Listar a frota com fabricante e categoria carregados.	
Resultado esperado	200 OK com os 12 veículos e seus relacionamentos.	



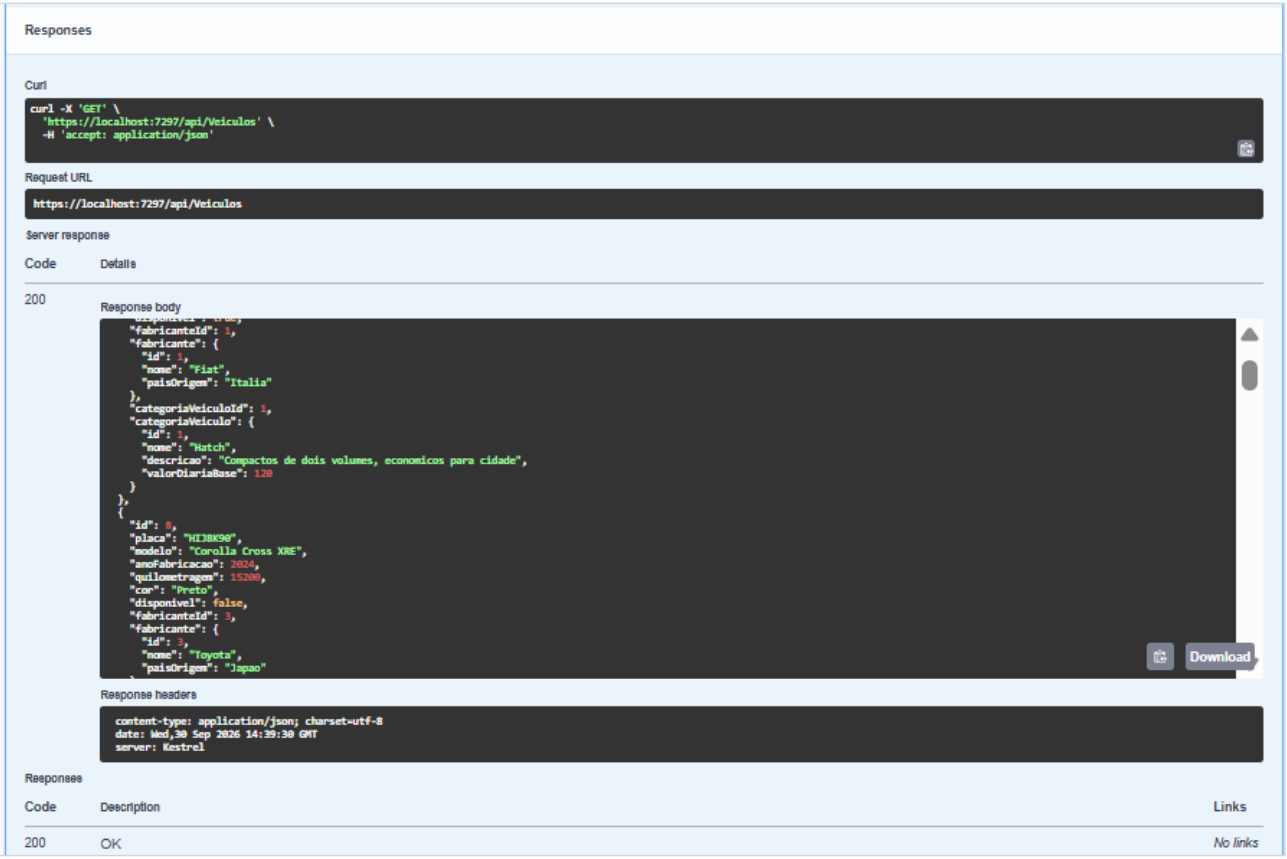
Evidência: 11-veiculos-get-200.png

12	POST	/api/Veiculos
Objetivo	Cadastrar um veículo vinculado a um fabricante e a uma categoria.	
Resultado esperado	201 Created; a placa é validada como única.	



Evidência: 12-veiculos-post-201.png

13	GET	/api/Veiculos/{id}
Objetivo	Consultar o veículo recém-criado.	
Resultado esperado	200 OK com fabricante e categoria aninhados.	



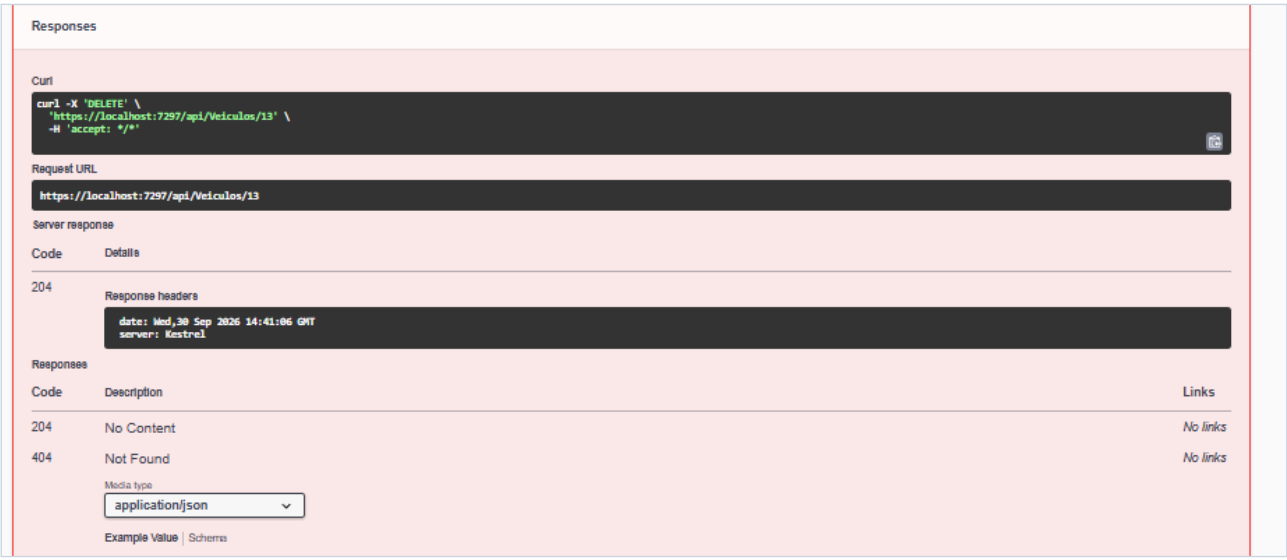
Evidência: 13-veiculos-getid-200.png

14	PUT	/api/Veiculos/{id}
Objetivo	Atualizar a quilometragem do veículo.	
Resultado esperado	204 No Content.	



Evidência: 14-veiculos-put-204.png

15	DELETE	/api/Veiculos/{id}
Objetivo	Excluir o veículo criado no teste 12.	
Resultado esperado	204 No Content — não possui aluguéis registrados.	

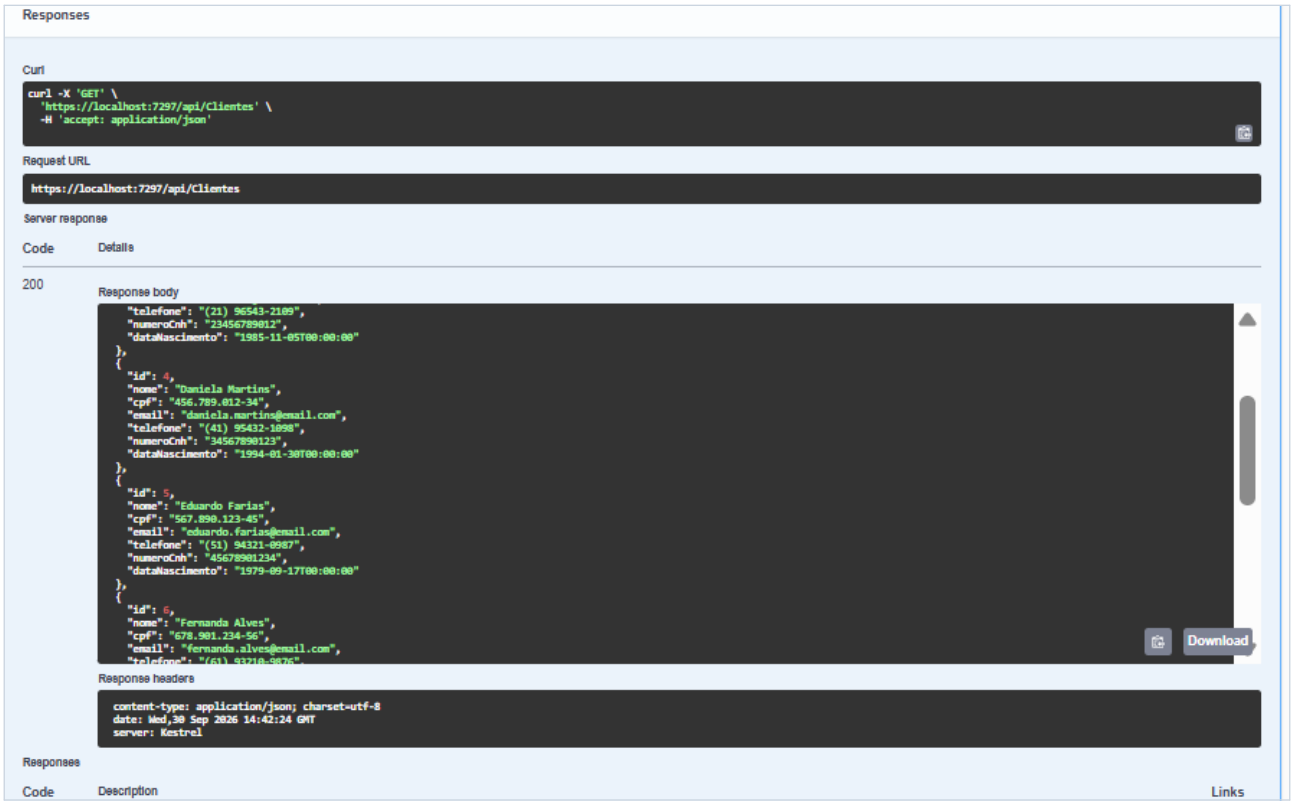


Evidência: 15-veiculos-delete-204.png

Clientes

CRUD completo da entidade Cliente.

16	GET	/api/Clientes
Objetivo	Listar todos os clientes.	
Resultado esperado	200 OK com os 8 clientes.	



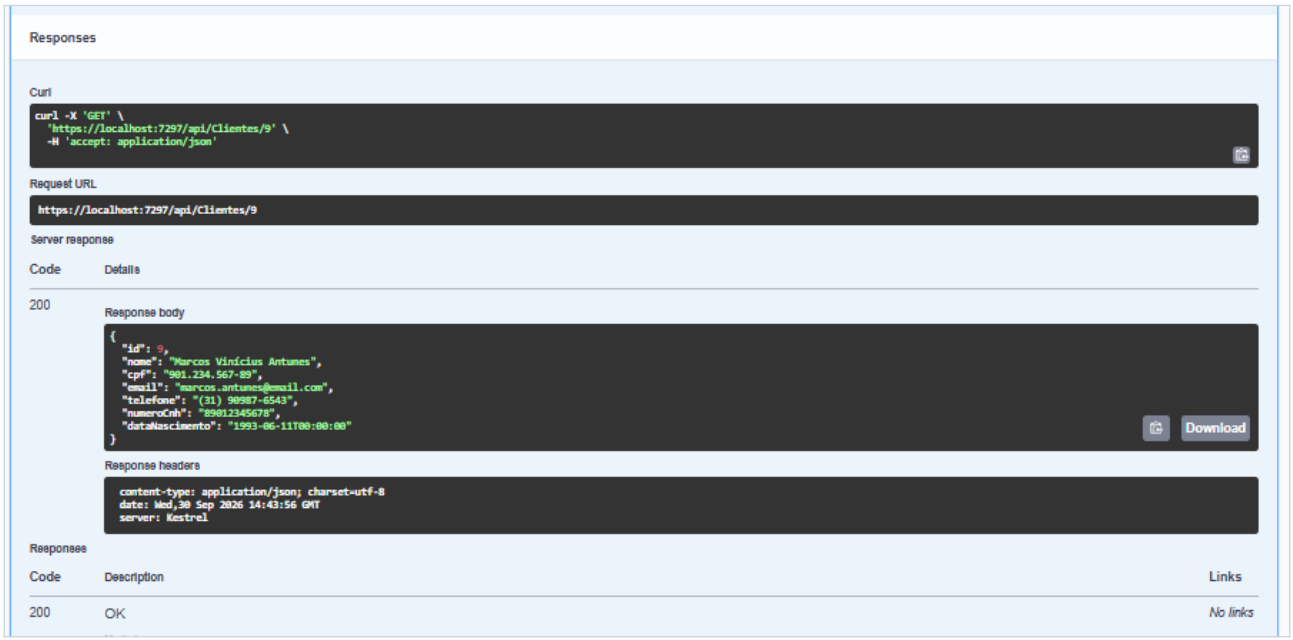
Evidência: 16-clientes-get-200.png

17	POST	/api/Clientes
Objetivo	Cadastrar um cliente com CPF e e-mail válidos.	
Resultado esperado	201 Created; o CPF é validado como único.	



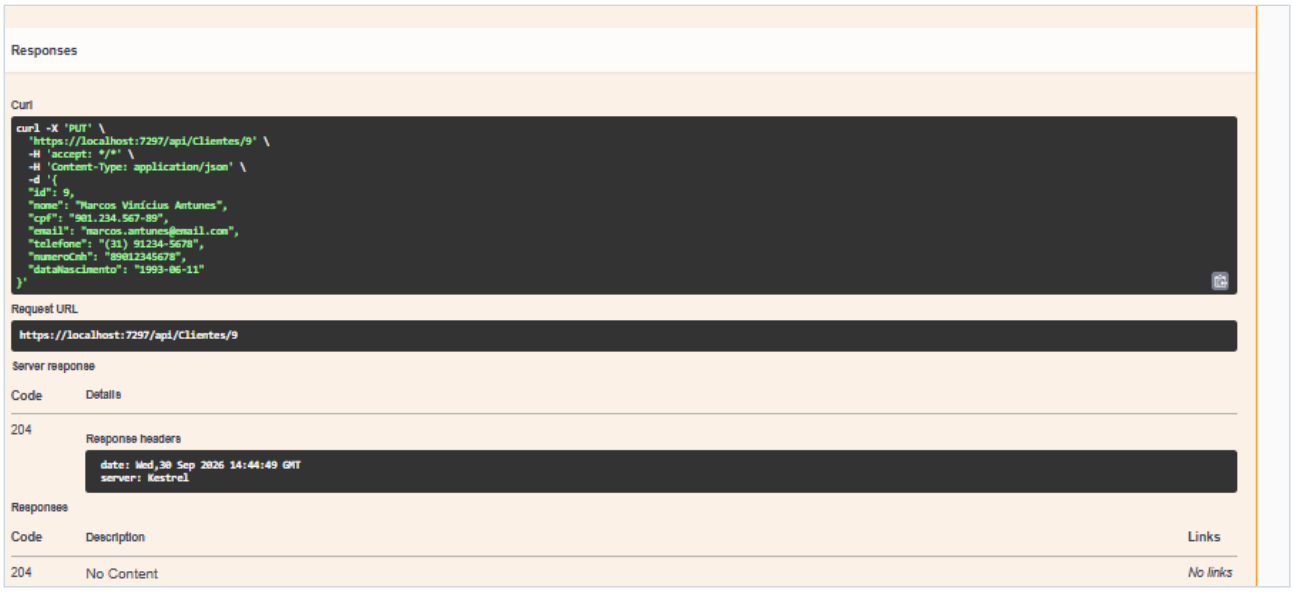
Evidência: 17-clientes-post-201.png

18	GET	/api/Clientes/{id}
Objetivo	Consultar o cliente recém-criado.	
Resultado esperado	200 OK com o registro.	



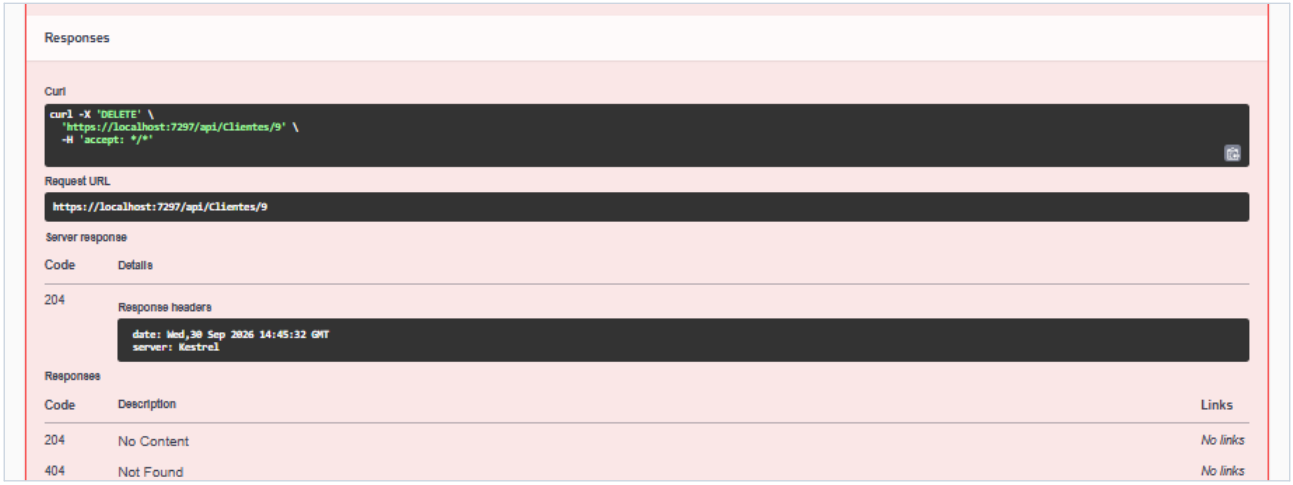
Evidência: 18-clientes-getid-200.png

19	PUT	/api/Clientes/{id}
Objetivo	Alterar o telefone do cliente.	
Resultado esperado	204 No Content.	



Evidência: 19-clientes-put-204.png

20	DELETE	/api/Clientes/{id}
Objetivo	Excluir o cliente criado no teste 17.	
Resultado esperado	204 No Content — sem aluguéis em seu nome.	

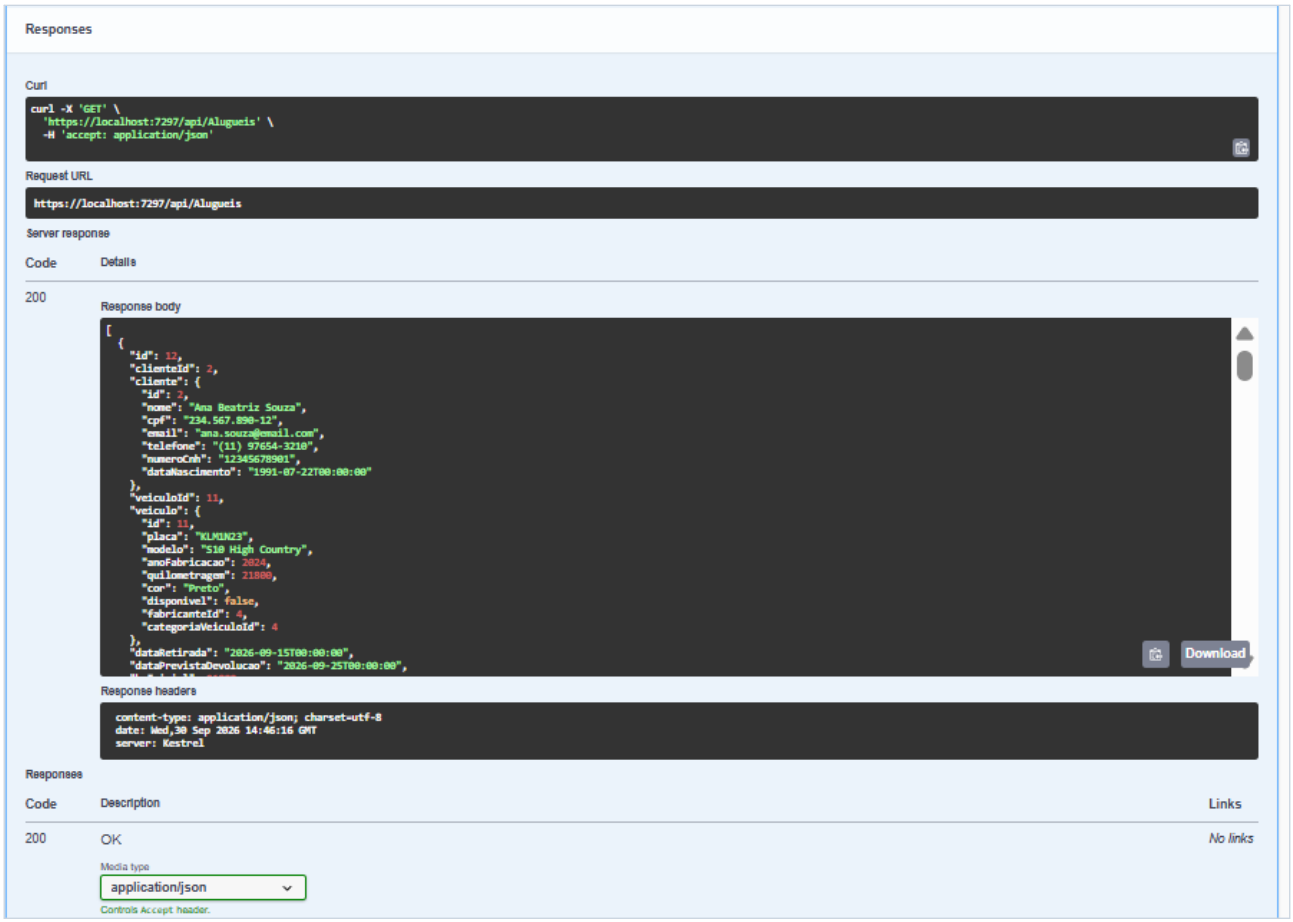


Evidência: 20-clientes-delete-204.png

Aluguéis

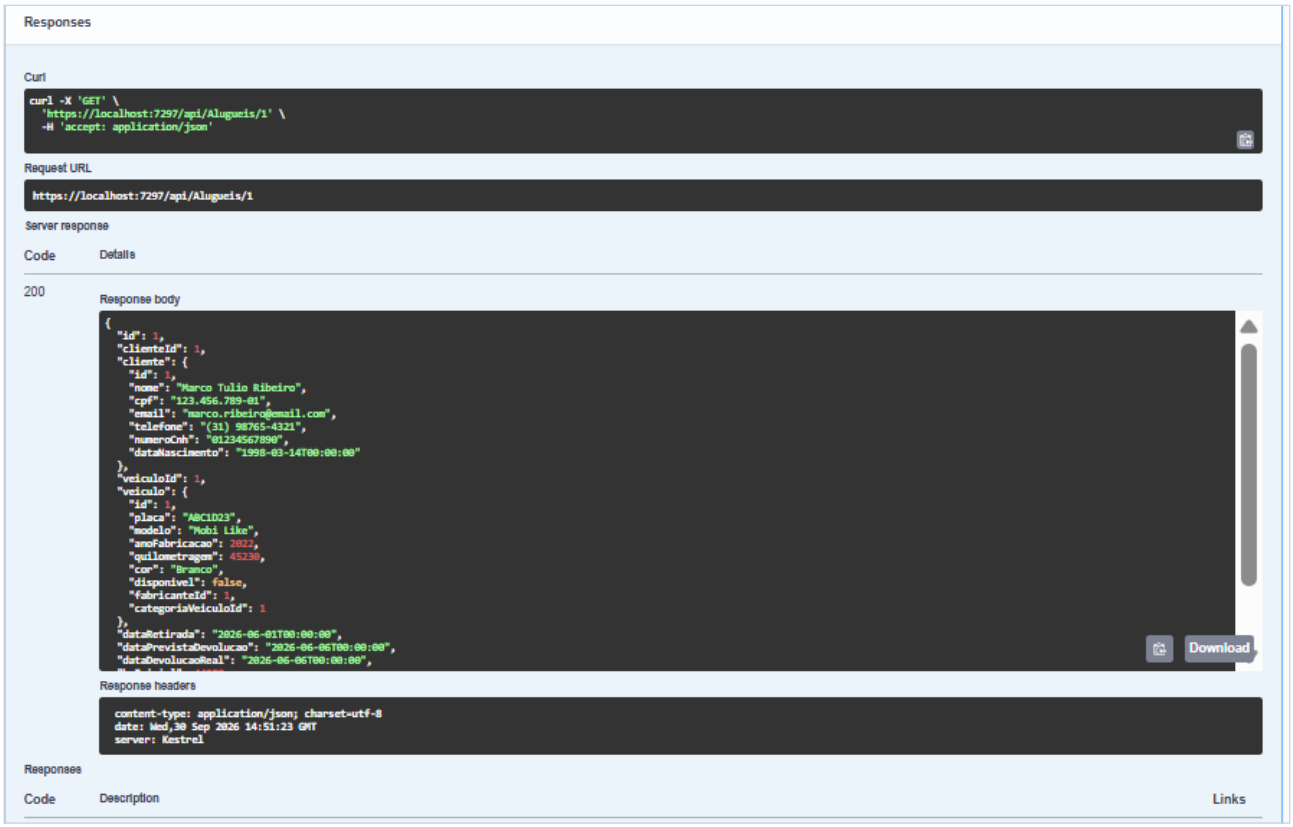
CRUD da entidade Aluguel e o registro de devolução do veículo.

21	GET	/api/Alugueis
Objetivo	Listar as locações, da mais recente para a mais antiga.	
Resultado esperado	200 OK com os 12 aluguéis, cliente e veículo carregados.	



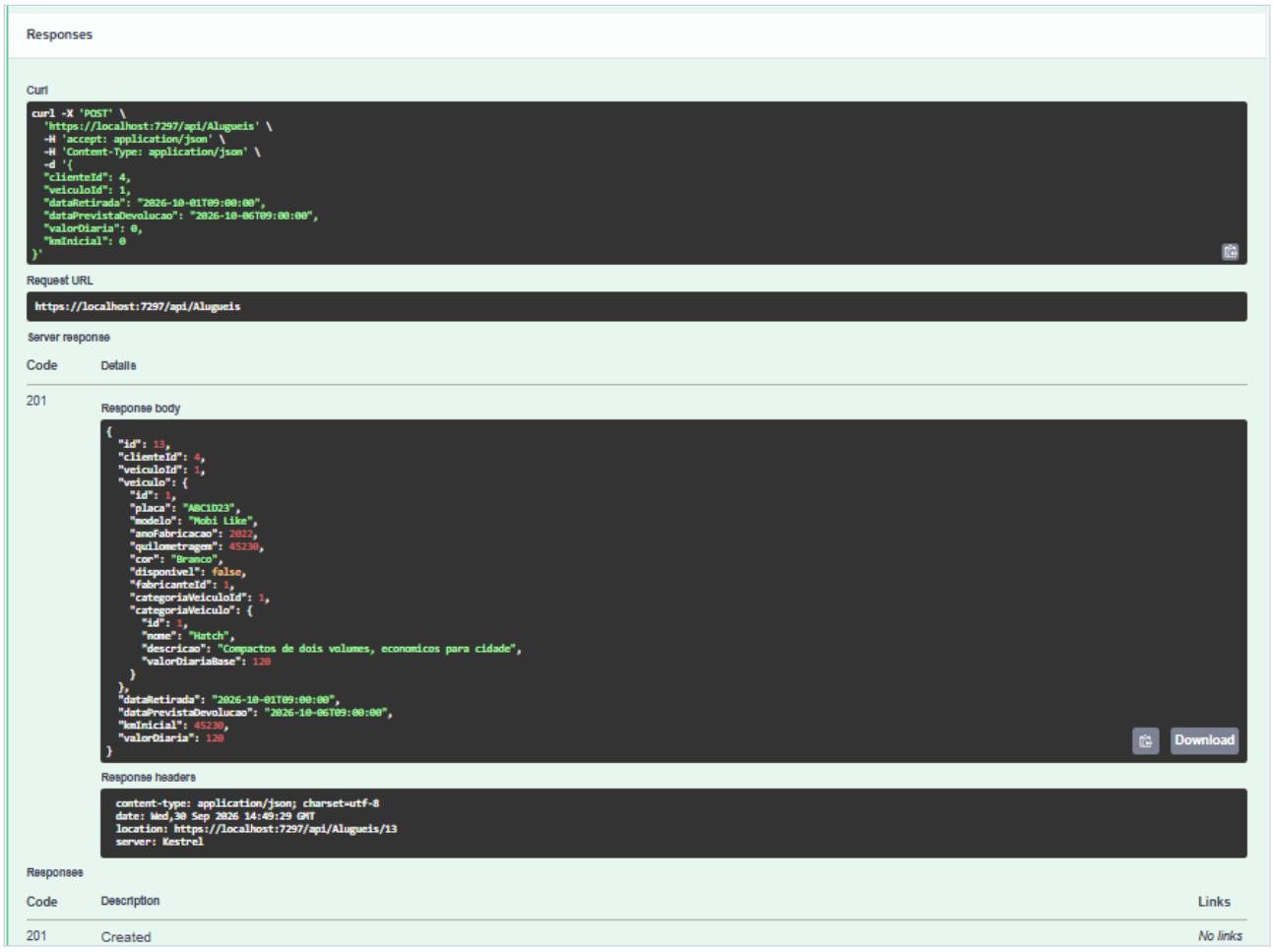
Evidência: 21-alugueis-get-200.png

22	GET	/api/Alugueis/{id}
Objetivo	Consultar uma locação específica.	
Resultado esperado	200 OK com os dados do cliente e do veículo.	



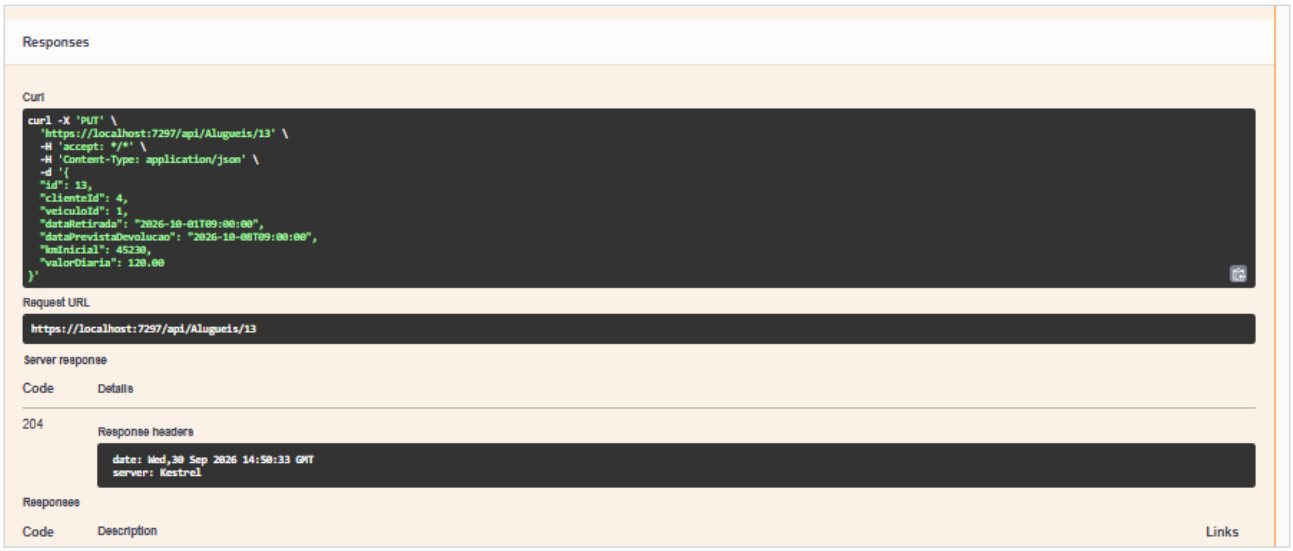
Evidência: 22-alugueis-getid-200.png

23	POST	/api/Alugueis
Objetivo	Registrar uma locação sem informar o valor da diária.	
Resultado esperado	201 Created; a API busca a diária vigente da categoria e marca o veículo como indisponível.	



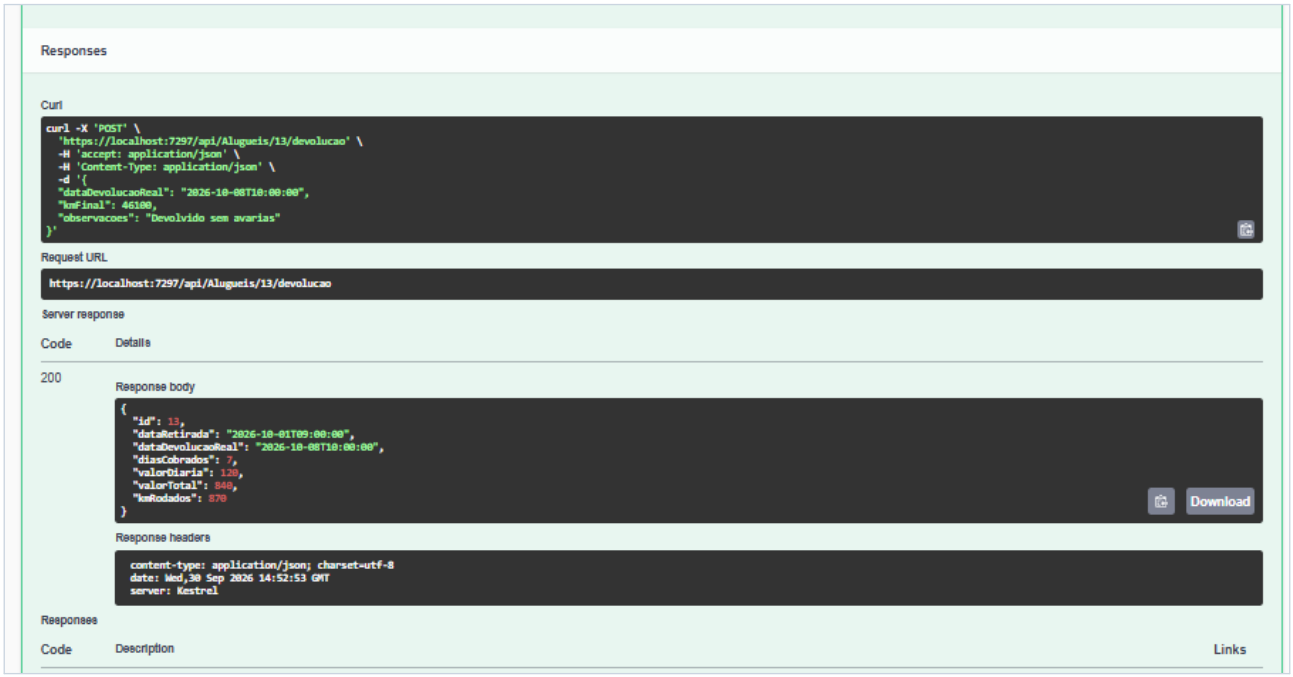
Evidência: 23-alugueis-post-201.png

24	PUT	/api/Alugueis/{id}
Objetivo	Estender o prazo previsto de devolução.	
Resultado esperado	204 No Content.	



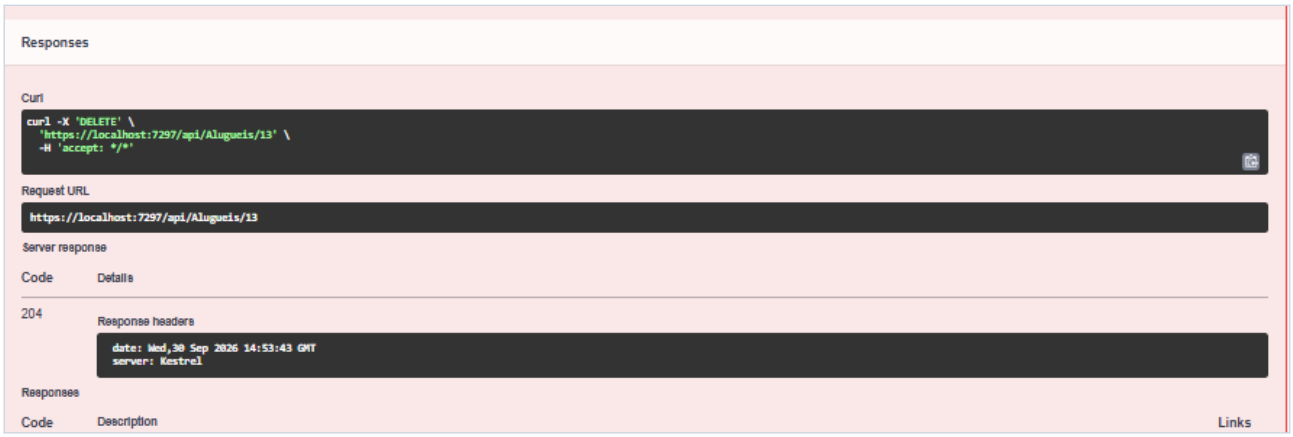
Evidência: 24-alugueis-put-204.png

25	POST	/api/Alugueis/{id}/devolucao
Objetivo	Registrar a devolução do veículo.	
Resultado esperado	200 OK com os dias cobrados, o valor total e os km rodados calculados; o veículo volta a ficar disponível.	



Evidência: 25-alugueis-devolucao-200.png

26	DELETE	/api/Alugueis/{id}
Objetivo	Excluir a locação criada no teste 23.	
Resultado esperado	204 No Content.	

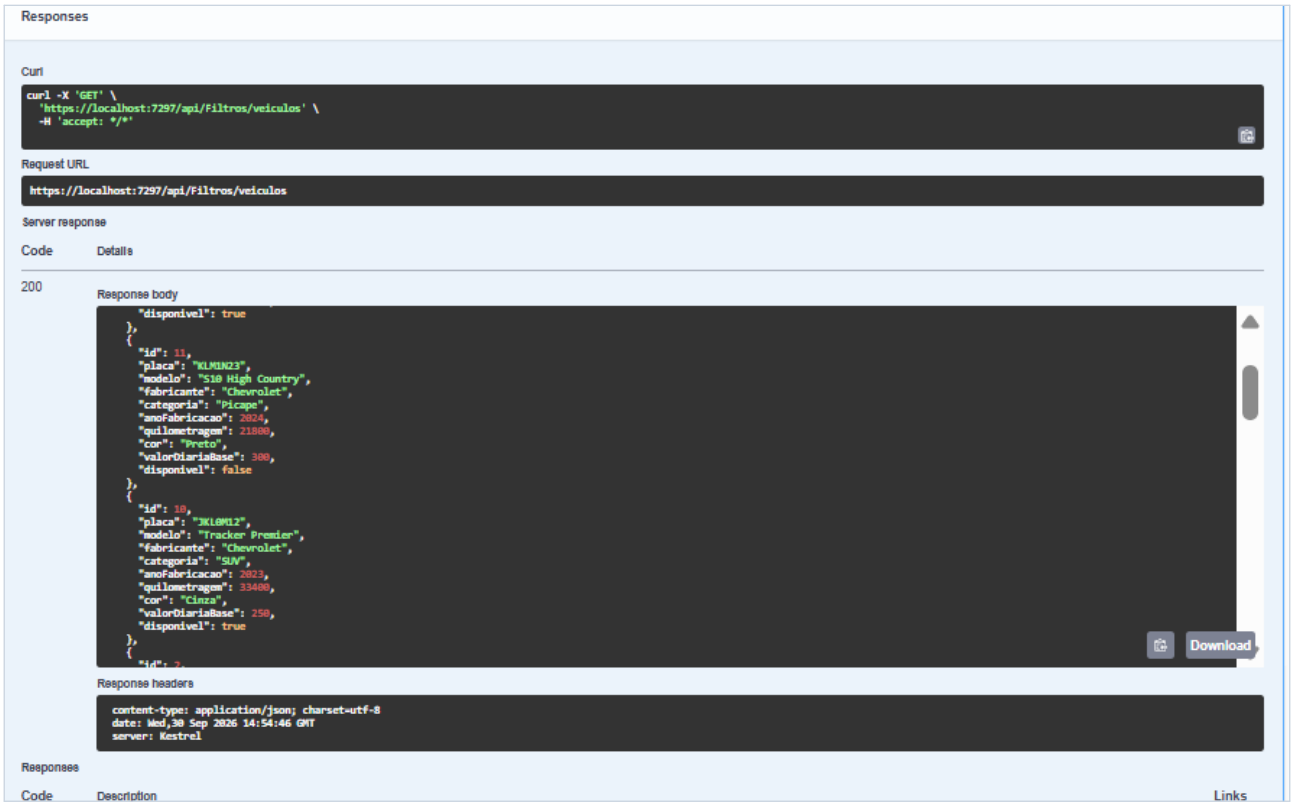


Evidência: 26-alugueis-delete-204.png

Filtros com junção de tabelas

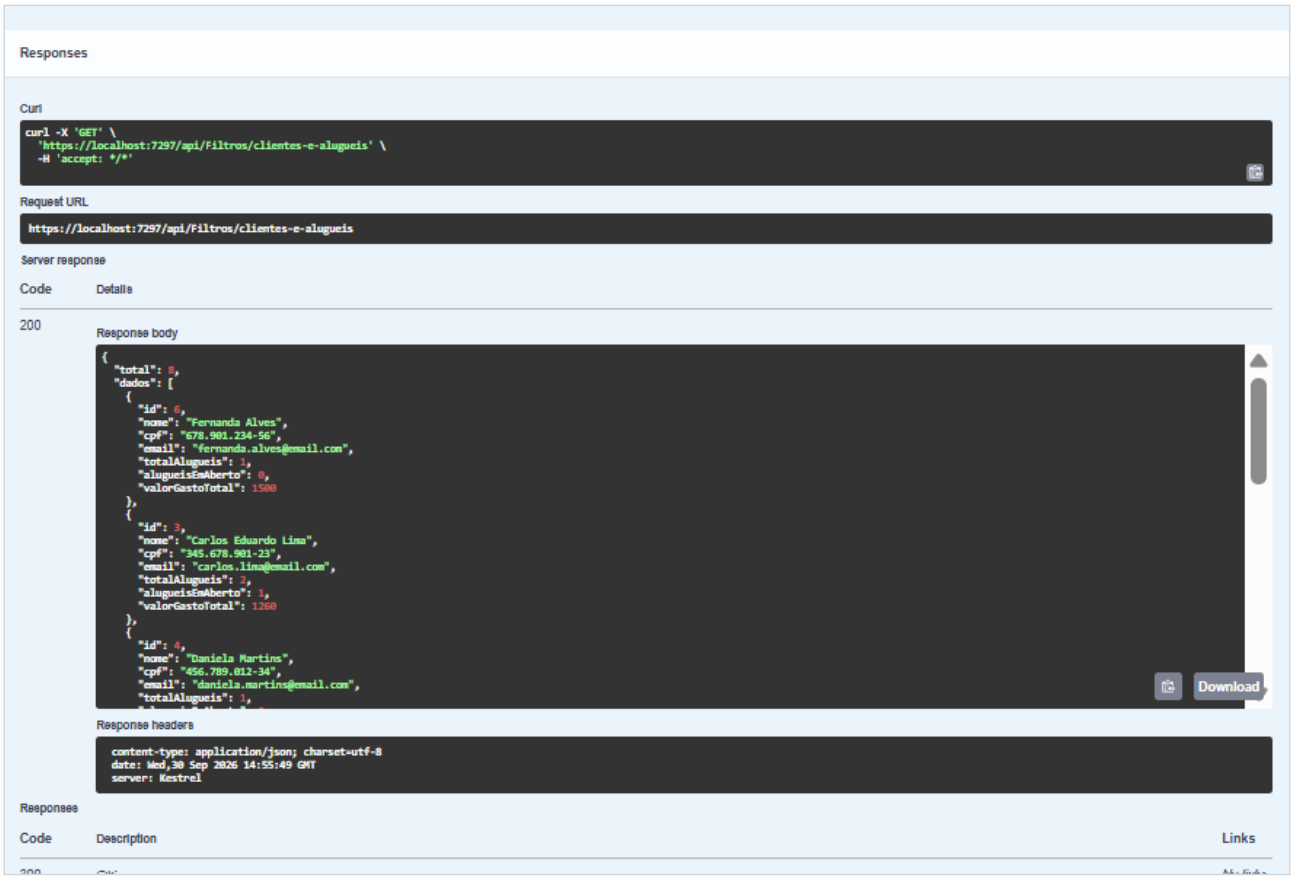
Seis consultas analíticas. Quatro usam INNER JOIN e duas usam LEFT JOIN, atendendo à exigência de pelo menos cinco filtros com dois tipos de junção distintos.

27	GET	/api/Filtros/veiculos
Objetivo	Frota completa com marca e categoria (INNER JOIN entre três tabelas).	
Resultado esperado	200 OK com os 12 veículos, cada um trazendo o nome do fabricante e da categoria.	



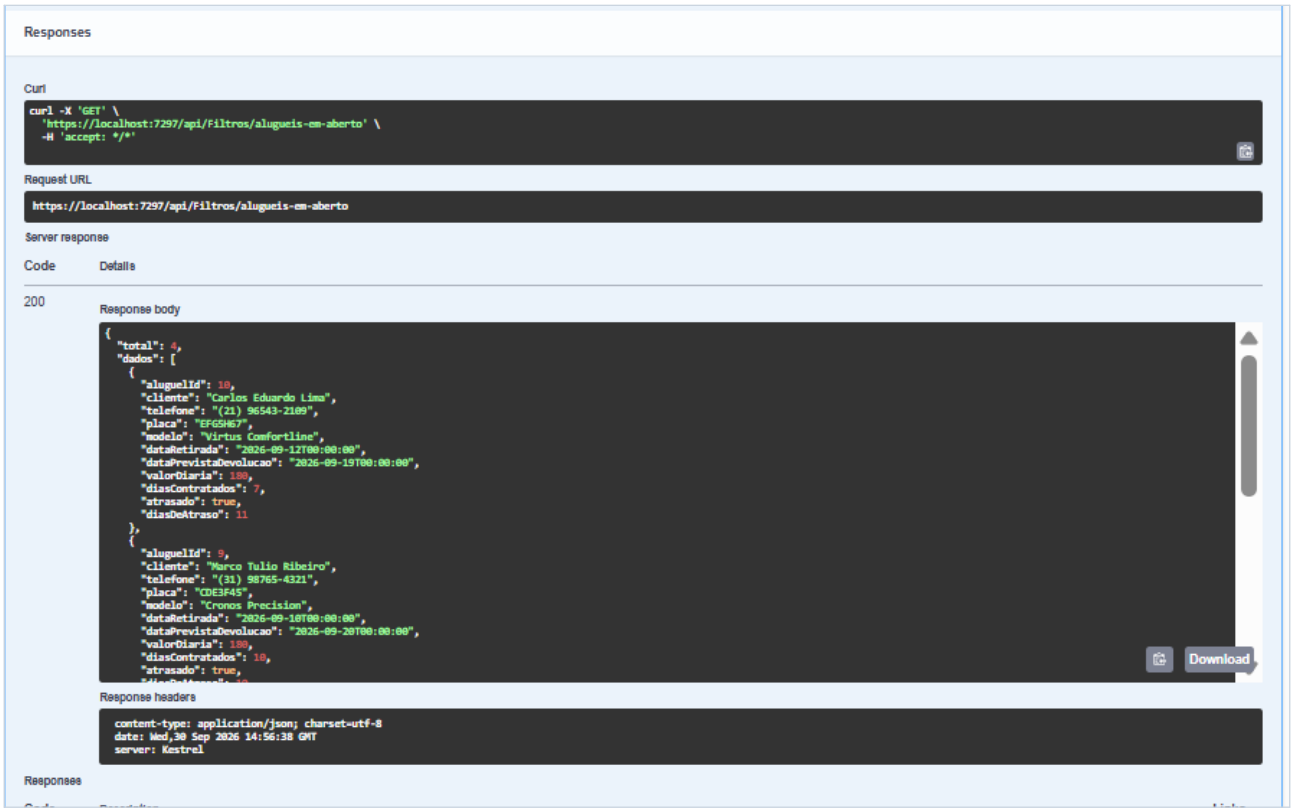
Evidência: 27-filtro-veiculos-200.png

28	GET	/api/Filtros/clientes-e-alugueis
Objetivo	Histórico por cliente, incluindo quem nunca alugou (LEFT JOIN).	
Resultado esperado	200 OK com os 8 clientes; os sem locação aparecem com totalAlugueis igual a zero.	



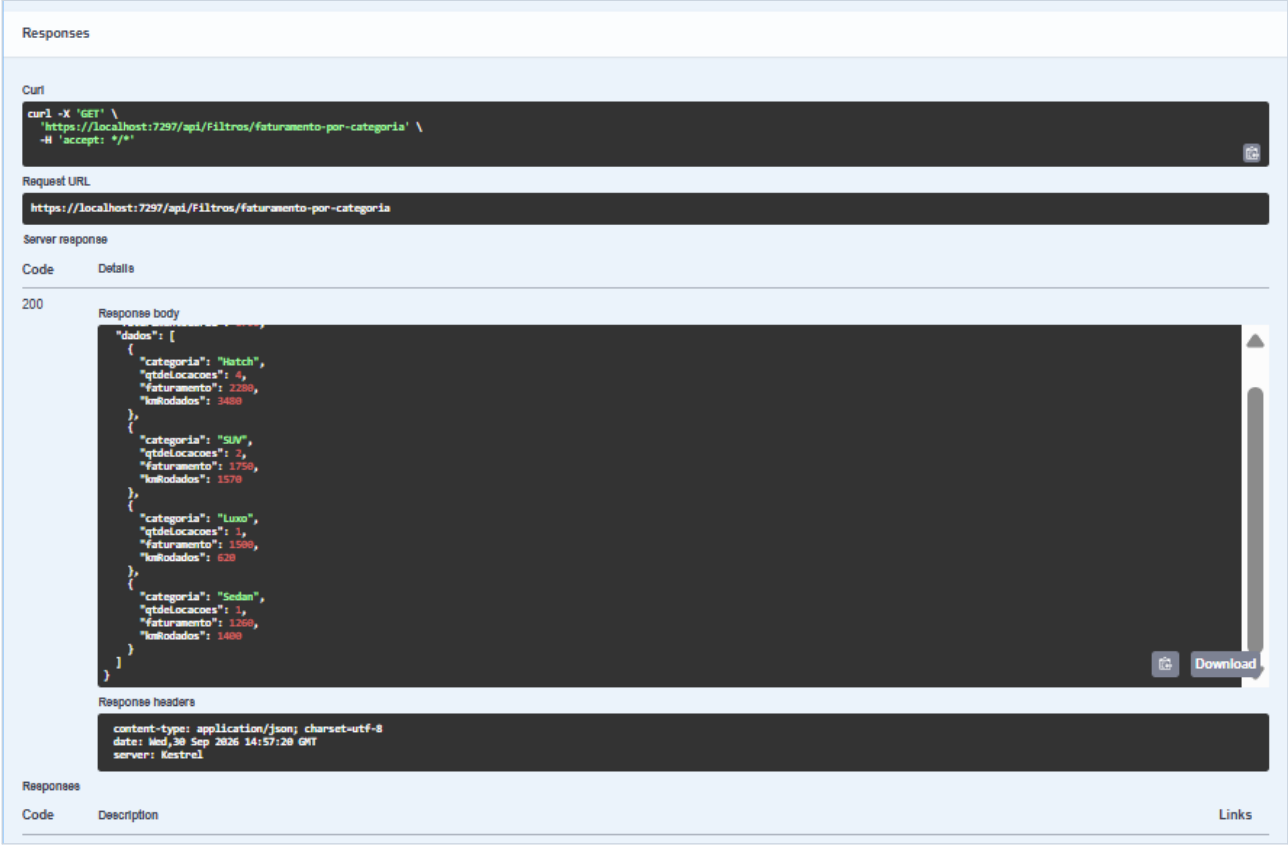
Evidência: 28-filtro-clientes-alugueis-200.png

29	GET	/api/Filtros/alugueis-em-aberto
Objetivo	Locações não encerradas, com contato do cliente (INNER JOIN entre três tabelas).	
Resultado esperado	200 OK com os 4 aluguéis sem devolução e o cálculo de dias em atraso.	



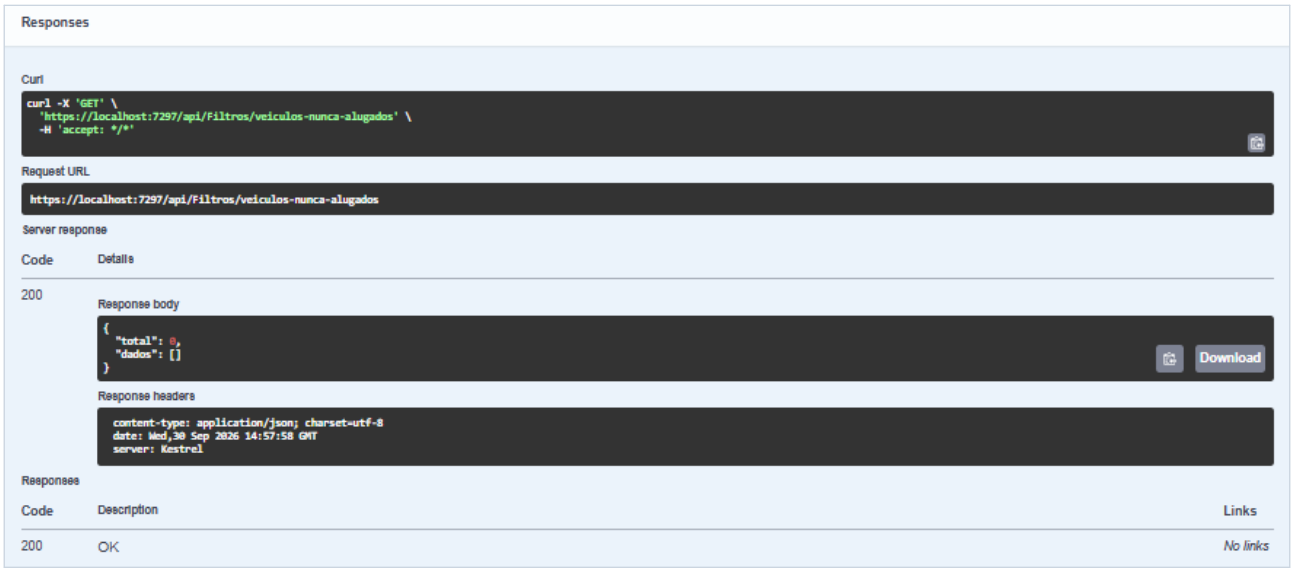
Evidência: 29-filtro-alugueis-aberto-200.png

30	GET	/api/Filtros/faturamento-por-categoria
Objetivo	Receita e quilometragem agrupadas por categoria (INNER JOIN com agregação).	
Resultado esperado	200 OK com o faturamento por categoria e o total geral.	



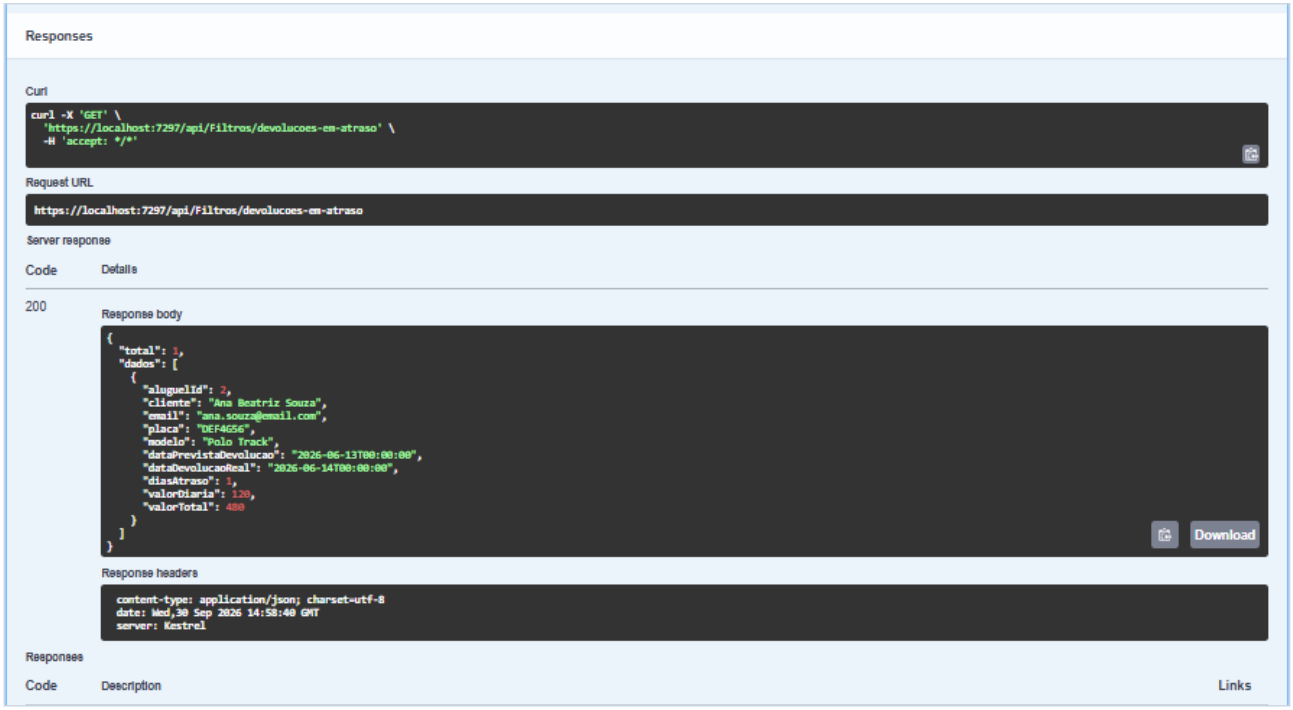
Evidência: 30-filtro-faturamento-200.png

31	GET	/api/Filtros/veiculos-nunca-alugados
Objetivo	Veículos ociosos (LEFT JOIN retendo apenas as linhas sem correspondência).	
Resultado esperado	200 OK com os veículos que nunca foram objeto de locação.	



Evidência: 31-filtro-nunca-alugados-200.png

32	GET	/api/Filtros/devolucoes-em-atraso
Objetivo	Devoluções entregues após o prazo (INNER JOIN entre três tabelas).	
Resultado esperado	200 OK com as locações atrasadas, ordenadas pelo maior atraso.	

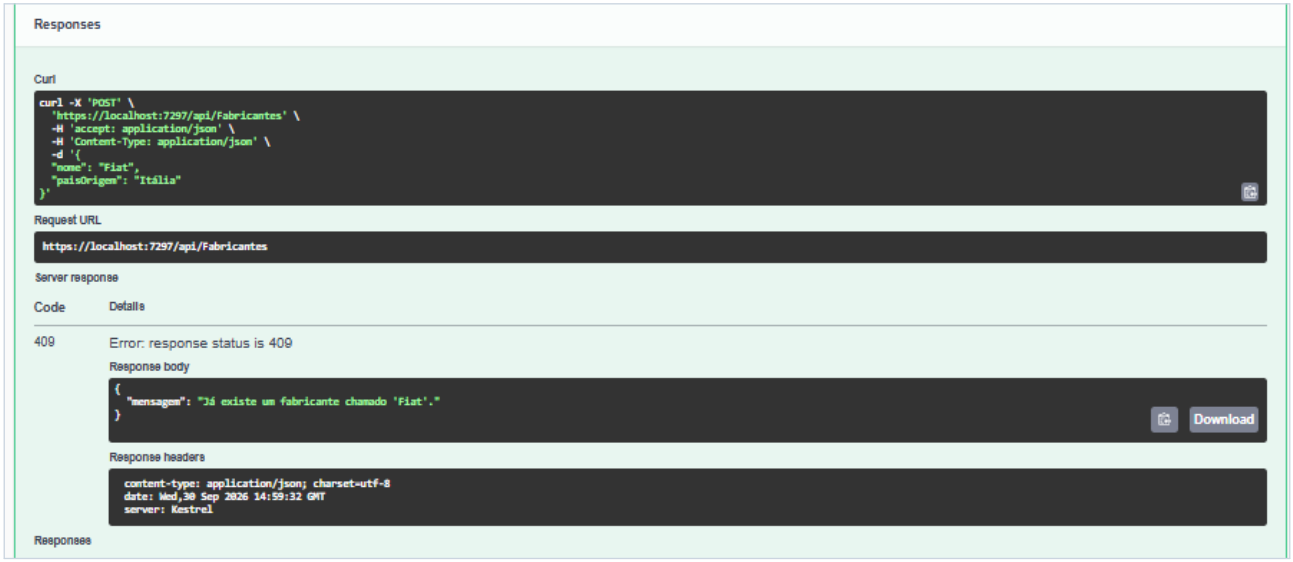


Evidência: 32-filtro-atrasos-200.png

Validação e tratamento de erros

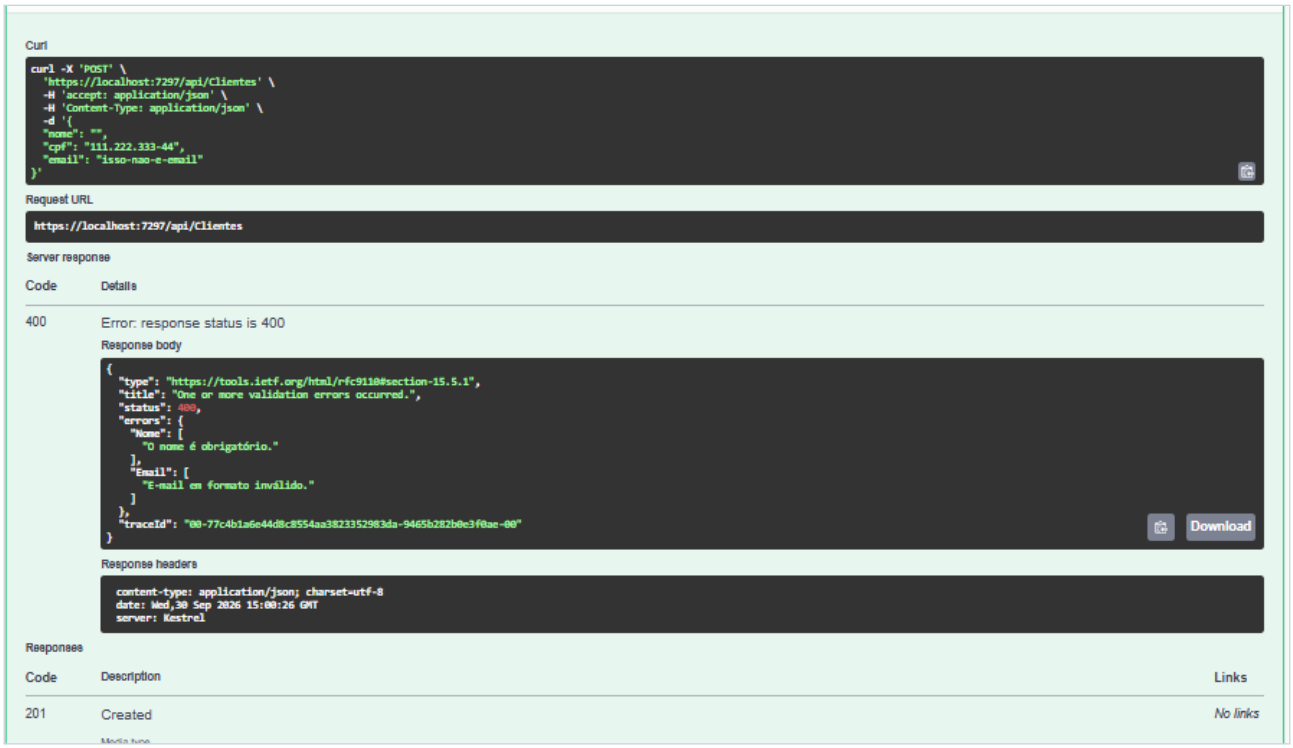
Testes que exercitam as três camadas de proteção: anotações de modelo, regras de negócio verificadas no controlador e o middleware global de exceções.

33	POST	/api/Fabricantes
Objetivo	Tentar cadastrar um fabricante com nome já existente.	
Resultado esperado	409 Conflict com mensagem explicando a duplicidade.	



Evidência: 33-erro-fabricante-duplicado-409.png

34	POST	/api/Clientes
Objetivo	Enviar um cliente com nome vazio e e-mail em formato inválido.	
Resultado esperado	400 Bad Request listando cada campo e a respectiva falha de validação.	



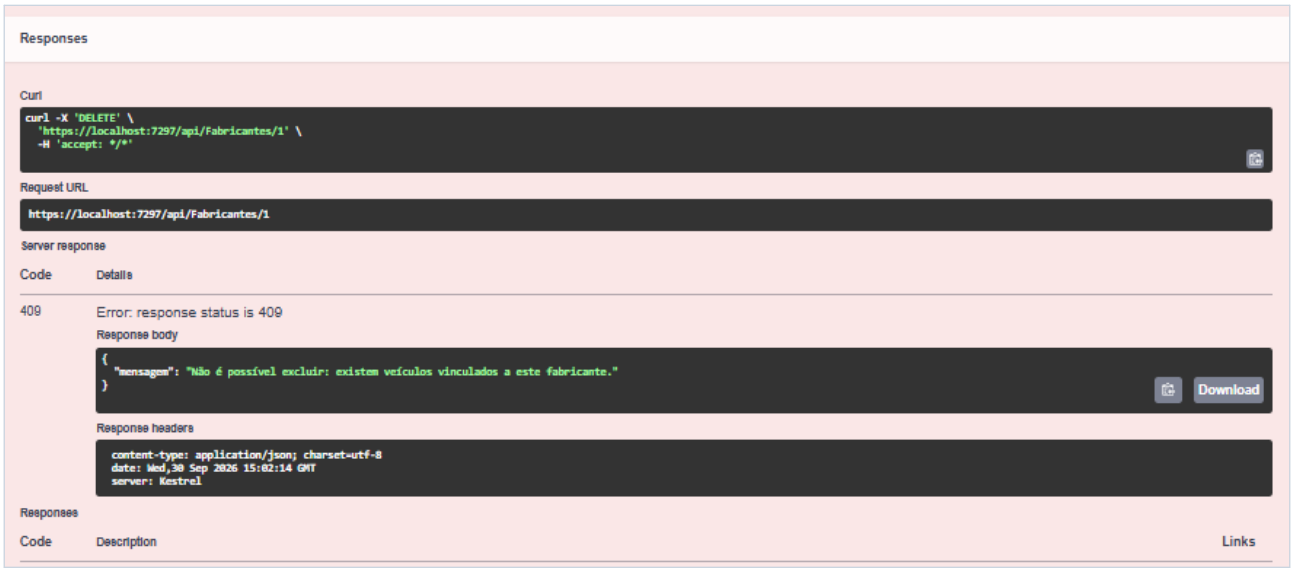
Evidência: 34-erro-cliente-invalido-400.png

35	GET	/api/Veiculos/999
Objetivo	Consultar um veículo com id inexistente.	
Resultado esperado	404 Not Found com mensagem clara, em vez de lista vazia.	



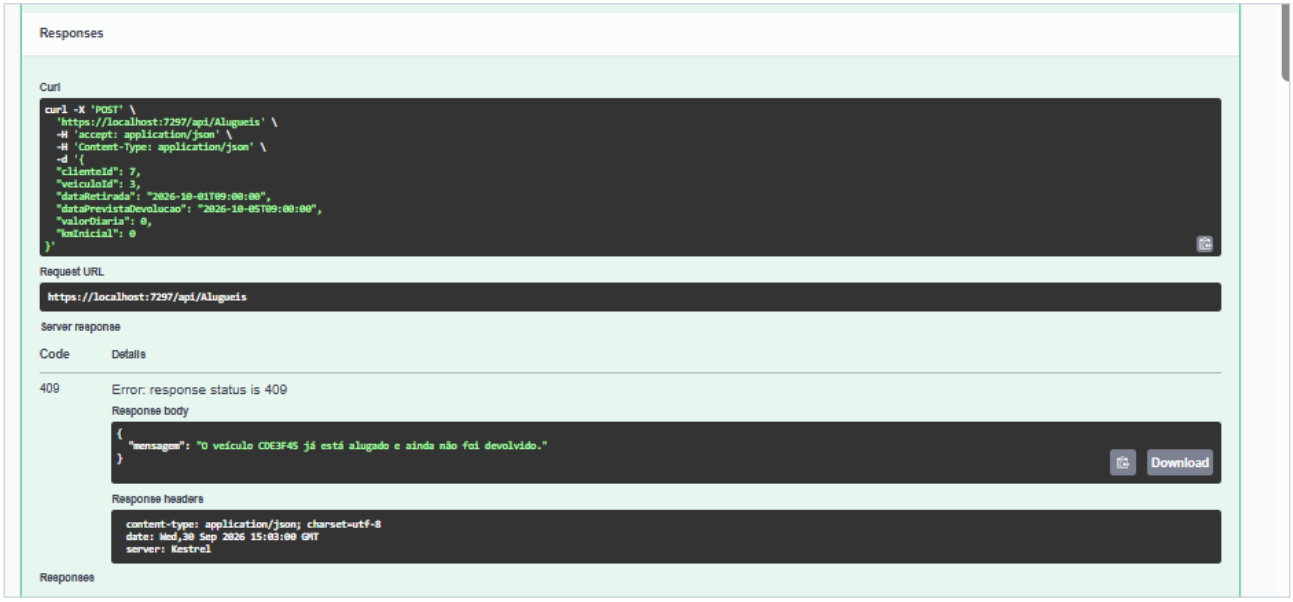
Evidência: 35-erro-veiculo-inexistente-404.png

36	DELETE	/api/Fabricantes/1
Objetivo	Excluir um fabricante que possui veículos vinculados.	
Resultado esperado	409 Conflict; a API bloqueia antes de o banco recusar, preservando a integridade referencial.	



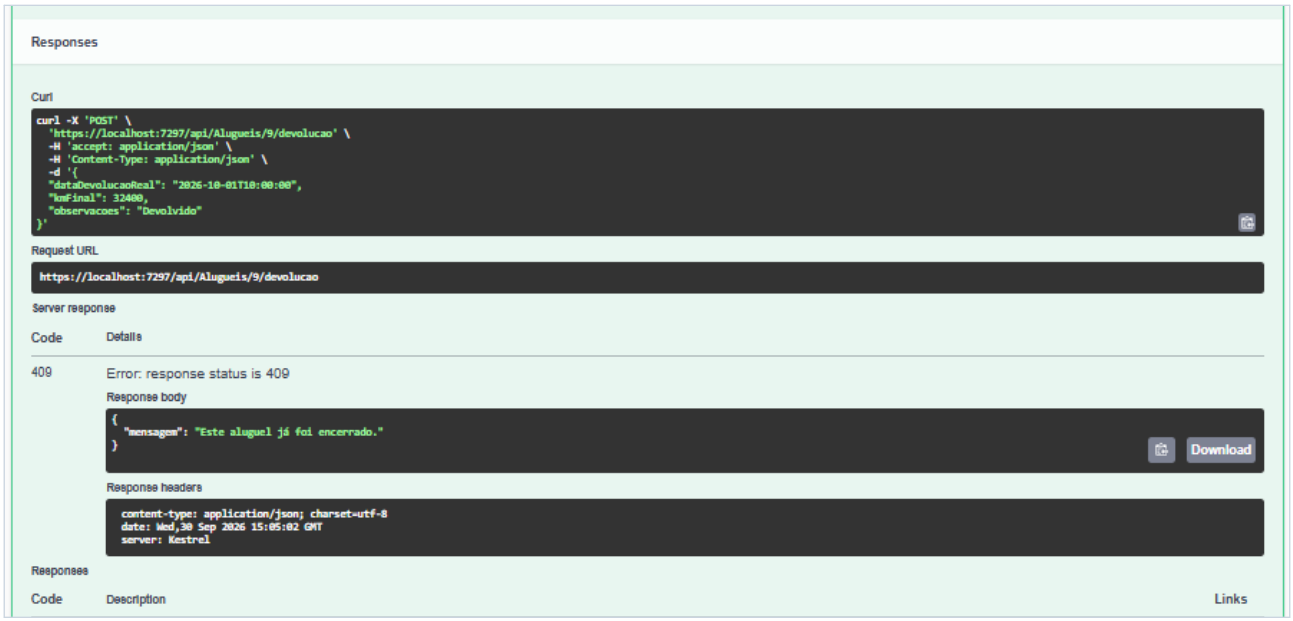
Evidência: 36-erro-delete-com-dependentes-409.png

37	POST	/api/Alugueis
Objetivo	Alugar um veículo que já está em uma locação em aberto.	
Resultado esperado	409 Conflict informando que o veículo não foi devolvido.	



Evidência: 37-erro-veiculo-ocupado-409.png

38	POST	/api/Alugueis/{id}/devolucao
Objetivo	Registrar a devolução de uma locação já encerrada.	
Resultado esperado	409 Conflict informando que o aluguel já foi encerrado.	



Evidência: 38-erro-devolucao-repetida-409.png